DISCOVERY READOUT · 2026-03-11

Woven Requirements Automation

Automating the intake and consolidation of software requirements from Toyota/CA sources into JAMA for Woven's SV&V team.

Team	Status	Pilot Scope	Sources
Dan Maser, Gus Schissler, Nathan Papes	In Progress	Lane Change SWRDs → JAMA	4 stakeholder interviews

The Problem

Requirements arrive scattered, change without warning, and are consolidated entirely manually.

CONTEXT

Two Requirements Problems at Woven

Top-Down

Requirements flowing from Toyota via the CA liaison team into Woven are scattered across tools, poorly structured, and change without any notification reaching the right teams.

Externally constrained, harder to solve. **Confirmed priority by Julia Pralle.**

Bottom-Up (Internal)

Autonomous driving software was written before formal requirements were established. Knowledge is tribal and scattered inside Woven.

Lower priority (internal to Woven, more available solutions). **Out of scope for this effort.**

PAIN POINT

Where Requirements Live Today

No single source of truth. No centralized visibility.

Git Repositories

SWRDs as markdown/YAML. Primary source for Lane Change. Not directly surfaced; accessed via PDM docs.

JAMA

Requirements management platform. Ground truth, but populated manually by copy-paste from Git.

Google Docs

PRD and URD documents from CA/PDM team. Nick reviews these daily alongside Git files.

Confluence

Additional documentation. No systematic connection to JAMA or change tracking.

Spreadsheets

URD spreadsheets from CA. Japanese + English descriptions; used for context only.

Tribal Knowledge

TBDs only resolvable by developers, who often don't know the answer either.

WHY THIS MATTERS

The Scale of the Problem

There are **hundreds of requirements** across many different functional areas, and every one is being handled manually today.

> Lane Change alone has 600+ SWRDs. Each must be found, read, interpreted, and entered into JAMA by hand, then tracked for changes, linked to test cases, and updated whenever Toyota revises them. There is no automated alert when something changes mid-quarter. There is no single place to see what's been processed and what hasn't.

The work doesn't have to be entirely manual.

USER RESEARCH

"All I do is copy things into JAMA."

Hannah Lane, Systems Engineer

Hyperbole, but telling. Said unprompted, before the interview even began.

USER RESEARCH

What We Heard: Four Interviews

Hannah Lane

Primary user with the most acute pain. Manages 600+ lane change SWRDs, manually copying them into JAMA and tracking versions by hand. Uses Wovey (LLM) for test case drafting at ~90% effectiveness.

- Opens PRD → URD → Git docs → manually creates JAMA entry
- Tracks changes by maintaining version numbers in JAMA entries
- No automated alert when Toyota updates requirements mid-quarter

Nick Carcione

Reviews SWRDs from three sources to assess usability. Rejects overly prescriptive requirements and feeds back to product/CA teams.

- Many SWRDs written for rules-based TSS4, not Woven's ML approach
- No way to pre-filter TBD-heavy or low-quality SWRDs before review
- Relies entirely on manual change detection

Andrea Zotelli

Reverse-engineering requirements from existing code. Struggling with scattered sources, Japan timezone barriers, and tribal knowledge gaps. Targeting production by early 2029.

- Asks developers to resolve TBDs; they often don't know either
- Searches Confluence, GitHub, Google Docs separately for every component

Julia Pralle

Manages the overall requirements flow. Implementing a formal approval phase and Change Requirement Board (CRB) for mid-quarter changes. Confirmed top-down intake as the priority.

- Requirements change "every single day" mid-quarter
- Needs intake progress visibility across capability groups
- CRB process needs automated change data to function

JOBS TO BE DONE

What the System Must Accomplish

Intake & Consolidation

Gather requirements from scattered sources and structure them into JAMA with full context and version tracking.

Change Detection & Notification

Detect when Toyota updates requirements and alert systems engineers before the impact is missed.

Test Case Generation

Auto-generate SWQT structure from approved requirements using LLM, reducing manual drafting from hours to minutes.

PHASE 1 BOUNDARIES

What's In and What's Out

✓ In Scope (Pilot)

- SYSTEM 1 Structured JAMA requirements entry from SWRD content
- SYSTEM 1 Gated test case (SWQT) generation via LLM
- SYSTEM 1 JAMA source linkage and version history
- SYSTEM 2 Lane change SWRDs from Git repository (pilot scope)
- SYSTEM 2 Automated SWRD discovery and batch import from PRD
- SYSTEM 3 Change detection via polling
- SYSTEM 3 Change notification to Hannah and systems engineers

✗ Out of Scope (Pilot)

- SBRDs and TPIDs (separate workflow)
- Multi-source consolidation beyond Git
- Scope translation: system-of-systems → IPA
- Integration with CA/Toyota upstream systems
- Bottom-up requirements backfilling from existing code

The MVP Solution

Automate the pipeline from Git-based SWRDs into structured JAMA entries, with change detection and gated test case generation.

INSIGHT

From Authoring to Reviewing

Hannah has shifted from manual authoring to reviewing and curating AI output.

> "A year ago it got me about 70% of the way there. Now I'd say it gets me about 90%."

> "I didn't type all of this by hand. It's not 2019 anymore."

Hannah Lane, Systems Engineer

Hannah already uses Wovey to generate test cases at ~90% effectiveness, then reviews and curates the output. She does less manual writing and more reviewing. This tool builds that workflow into the process rather than leaving it to individual initiative.

CONSTRAINTS

What We Can't Change

CA Authorship Is Out of Scope

Hannah and Nick can highlight issues with incoming requirements, but cannot fix them at the source. The quality of what CA writes is outside Woven's control.

We're Not Changing the PRD Process

The tool slots into the existing PRD-driven intake flow. It does not redesign how requirements are scoped, prioritized, or handed off. That process stays as-is.

Teams Won't Move Their Docs

Established practices exist for where requirements live. The solution must work with existing sources, not ideal organization.

Process Still Being Defined

Woven is implementing the CRB process now. The target workflow is not yet stable, which is an opportunity to help shape it.

PROPOSED SOLUTION

Three-System Architecture

The solution has three distinct parts, which can be built iteratively.

SYSTEM 1 · FIRST PRIORITY

✍️

Requirements Creation

Takes raw SWRD content and structures it into plain-English, JAMA-ready entries, with a gated test case draft. We have a proof of concept working.

✓ **POC built, no access dependencies**

SYSTEM 2

📡

Automated Gathering

Reads the quarterly PRD, discovers all in-scope SWRDs, and batch-imports them, replacing the manual one-by-one hunt across Git and Google Docs.

Needs: Git + Google Drive access

SYSTEM 3

🔔

Change Management

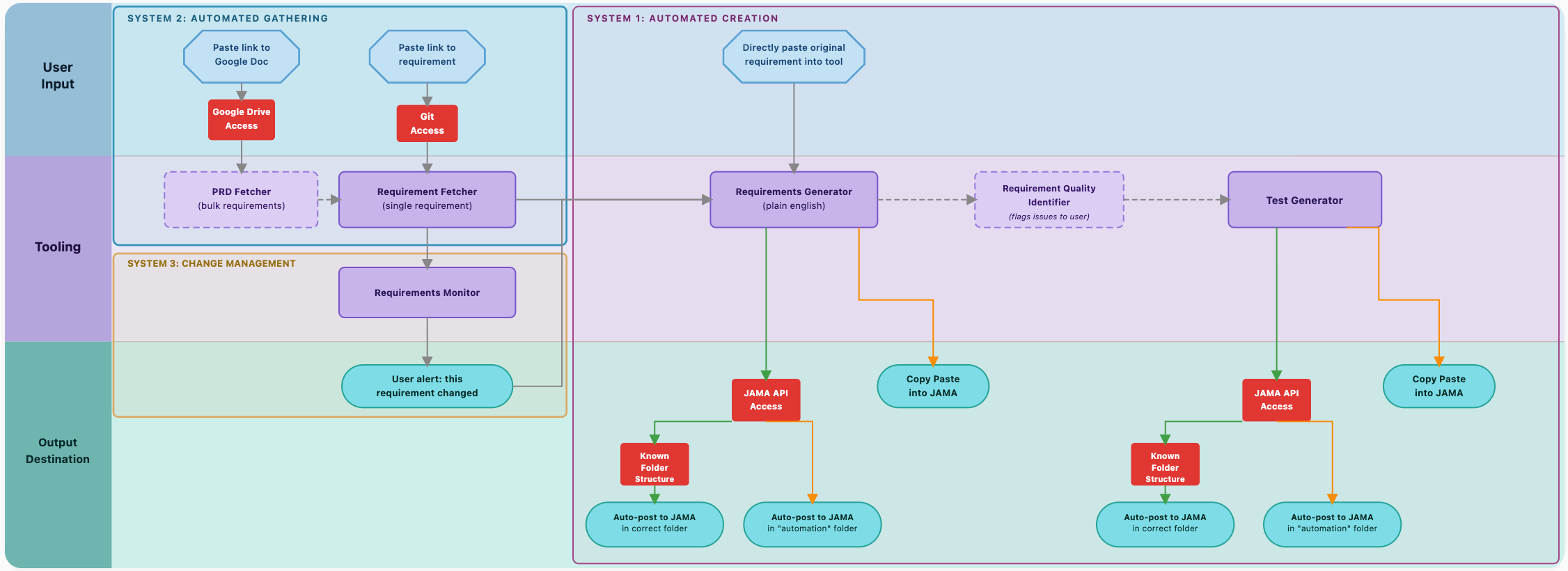
Watches for Toyota requirement updates mid-quarter and alerts the right engineers before the impact is missed. Extends System 2's access layer.

Needs: System 2 access + polling cadence

Each system is independently useful. System 1 works today with no access dependencies. Systems 2 and 3 unlock progressively as tooling gets access to Git, Google Drive, and JAMA's API.

PROPOSED SOLUTION

How the Systems Connect



SYSTEM 1 · PRIORITY

Requirements Creation

Turns raw SWRD content into structured, plain-English JAMA entries, and drafts SWQTs (test cases) once a requirement is approved. Both go into JAMA.

ACCESS TIERS: HOW MUCH AUTOMATION IS POSSIBLE

✓ WORKS TODAY

1. Paste → Review → Copy

User pastes raw SWRD text. Tool generates a plain-English SWRD entry + SWQT draft. User copies both outputs into JAMA manually.

No access dependencies

WITH JAMA API

2. Auto-Post to "Import" area

Tool writes both the SWRD entry and SWQT directly (if desired) into JAMA without copy-paste. Posts to a holding "automation" folder for review before moving to the right location.

Needs: JAMA API access

WITH JAMA API + FOLDER MAP

3. Auto-Post to Correct Location

With known folder structure, both the SWRD entry and SWQT route directly to their correct locations in JAMA, with no manual filing required.

Needs: JAMA API + known folder structure

SYSTEM 1

Open Questions

Copy-paste automation or intelligent transcription? Is the tool's job to faithfully move SWRD content into JAMA as-is (just faster), or should it actively improve what lands there: translating jargon, reformatting for readability, filling in structure? Hannah currently does this transformation manually, but she may prefer to keep that control rather than trust an LLM to interpret intent.

🟡 CONFIRM WITH HANNAH; TEST WITH SYSTEM 1 PROTOTYPE

Do we even need a front end? Is this a standalone UI, a CLI, or something that lives inside JAMA as a module? Julia has flagged a preference for fewer tools. If JAMA supports extensions, building inside JAMA rather than alongside it may be the right call.

🟡 OPEN; EXPLORE JAMA CUSTOM MODULE CAPABILITY BEFORE BUILDING A UI

SYSTEM 2

Automated Gathering

Instead of hunting through Git and Google Docs one requirement at a time, the tool reads the quarterly PRD and batch-imports every in-scope SWRD at once.

ACCESS TIERS: WHAT UNLOCKS AT EACH LEVEL

WITH GIT ACCESS

1. Single SWRD Fetch

User pastes a Git link to a specific SWRD. Tool fetches the content directly from the repo and queues it for System 1, without manually copying from GitHub.

Needs: Git read access

WITH GIT + GOOGLE DRIVE ACCESS

2. User-Initiated Batch

User pastes the PRD link. Tool reads the PRD from Google Drive, follows every SWRD link into Git, and queues them all for System 1, covering a full quarter's intake in one action.

Needs: Git + Google Drive read access

WITH DRIVE WEBHOOK OR POLLING

3. Auto-Triggered on PRD Approval

When the PRD moves to "approved" in Google Drive, batch import kicks off automatically with no user action needed. Cadence matches the quarterly release cycle.

Needs: Drive webhook or polling + Git access

Key dependency: System 3 is an extension of this; it uses the same access layer to watch for changes. Git and Drive access is the key dependency for both Systems 2 and 3.

SYSTEM 2

Open Questions

Can an automated agent access Git and Google Drive? This is distinct from whether we have access: what can a deployed tool or agent access inside Woven's environment? What's the approval process for granting a system (not a person) read access to internal repos and Drive?

🔴 BLOCKING. NO INFRASTRUCTURE ACCESS CONFIRMED YET.

How do we get access to new PRDs each quarter? The PRD is a Google Doc that gets replaced every quarter. The tool needs a way to discover and read each new one automatically, not just the current one we can find manually.

🔴 BLOCKING FOR BATCH IMPORT

Does Wovey already have access to Git? If Wovey is already deployed at Woven and has access to Git / Documentation, that's a single integration point that sidesteps every individual access request. Who does Julia point us to?

🟡 OPEN. WORTH ASKING BEFORE REQUESTING ACCESS SEPARATELY.

SYSTEM 3

Change Management

Watches Toyota's SWRD files for mid-quarter updates and alerts the right engineers before the impact is missed.

🔍

How It Works

A background job polls the Git repository on a set cadence (nightly or weekly). When a SWRD file changes, it diffs old vs. new and surfaces what specifically changed: interface, specification, or reason section.

Engineers get a digest: which SWRDs changed, what changed, link to the diff. The change can re-trigger System 1 processing for the affected requirement.

⚙️

Access Dependencies

System 3 extends System 2's access layer. It uses the same Git integration to watch for changes rather than batch-import them. No new access is required beyond what System 2 establishes.

Polling cadence and notification channel (Slack, email) are open questions to confirm.

Scope may narrow depending on what Reqtify already covers →

SYSTEM 3

Open Questions

Does Reqtify's change tracking cover enough that we don't need to build System 3? Reqtify handles suspicious-link detection and change tracking across 100+ connectors. Before fully scoping System 3, get a demo. If Reqtify already covers this well and isn't expensive, System 3 may narrow significantly.

🟡 GET A REQTIFY DEMO BEFORE SCOPING SYSTEM 3

What polling cadence should the tool have? Julia said requirements change "every single day" mid-quarter, but nightly alerts may create noise. Nightly, weekly, or something else?

🟡 CONFIRM WITH JULIA & TEAM; THIS AFFECTS INFRASTRUCTURE LOAD AND ALERT FATIGUE

What notification channel? Where does the change digest go: Slack, email, or surfaced directly inside JAMA? Who needs to be notified (Hannah, systems engineers, developers) and at what point in the change?

🟡 OPEN. NO CHANNEL DECISION MADE YET.

Where Does Reqtify Fit?

Julia flagged Reqtify as a tool to look into. Based on initial research, it solves a different but complementary problem. We recommend getting a demo before fully scoping System 3.

Pain Point	Our Tool	Reqtify
Copy SWRDs from Git → JAMA manually	**Solves (automated ingestion + structuring)**	**Does not solve (links existing entries; doesn't create new ones)**
Translate raw SWRDs to plain English	**Solves (LLM translation)**	**Does not solve (no content transformation)**
Generate SWQT test cases from SWRDs	**Solves (LLM generation)**	**Does not solve (traces existing tests; doesn't generate new ones)**
Detect when upstream SWRDs change	**Partially solves (requirements monitor)**	**Solves well: suspicious link detection, change tracking across 100+ connectors**
Trace requirements → tests → implementation	**Does not solve (not in scope)**	**Solves well (this is its core purpose)**
Coverage / impact reporting	**Does not solve**	**Solves well**

PROOF OF CONCEPT FOR SYSTEM 1

What We've Built

A working System 1 prototype: paste in a raw SWRD, get plain-English output and a test case draft, ready to copy into JAMA. **We'll demo this live.**

Live demo

Where We Go Next

System 1 is working. The questions below determine how fast we can build Systems 2 and 3.

NEXT STEPS

Immediate Actions

1. **Confirm alignment on System 1**

   Based on the demo: does the approach match what the team needs? Confirm the translation model, output format, and whether a standalone UI is the right call before building further.

2. **Usability test with Hannah**

   Validate the translation approach and interface model before locking in System 1's final form.

3. **Clarify JAMA API access and folder structure**

   Key dependency for Systems 2 and 3. Who owns JAMA admin access at Woven?

4. **Ask Julia about building a project for Wovey API**

   Wovey allows for project creation and self-onboarding, we just need details for the inputs.

5. **Get a Reqtify demo**

   Before fully scoping System 3, see what Reqtify handles. If it covers change-linking, System 3's scope narrows significantly.

AREAS TO EXPLORE

What We Need to Figure Out

JAMA API access and folder structure: What API access is available, and what is the folder/container structure we'd need to write into? Is there any JAMA widget or extension capability we could build into rather than a separate tool?

🔴 KEY DEPENDENCY FOR SYSTEMS 2 AND 3

System access for the AI: What systems (Git, Google Drive, JAMA) can an automated agent access inside Woven's environment? What approval process exists for this? Who owns it?

🔴 KEY DEPENDENCY FOR SYSTEMS 2 AND 3

Usability test with Hannah: System 1 is working. We need to sit down with Hannah to validate the translation approach, interface model, and whether plain-English output is actually what she wants, or if she'd prefer structured-but-untranslated.

🟡 NEXT STEP. TEST BEFORE BUILDING FURTHER.

Reqtify demo: Before fully scoping System 3, it's worth seeing what Reqtify can and can't do firsthand. If it handles traceability and change-linking well, System 3's scope narrows significantly.

🟡 OPEN. GET A DEMO, THEN DECIDE.